

Universidad Técnica Estatal de Quevedo (UTEQ)

Facultad de Ciencias de la Computación

Carrera: Ingeniería en Software (Rediseño)

Asignatura: Aplicaciones Web

Período académico: 2026-2027 PPA

Título del PFC:

**Sistema de Gestión de Recursos Operativos,
Administrativos y de Seguridad para Cooperativas de
Transporte Nacional (SGROAS)**

Entrega 1B: Autenticación JWT + CRUD

Integrantes (CET):

- Castro Espinoza Kevin Moisés** C.I: 1250907019
- Escudero Plaza María del Rosario** C.I: 0957401847
- Tejada Bajaña Luis Alejandro** C.I: 1207432939

Docente: Ing. Guerrero Ulloa Gleiston Ciceron

Fecha: 20 de junio de 2026

URL del repositorio: [Alxjandr07/SGROAS-ProyectoAppWeb at entrega-1b](https://github.com/Alxjandr07/SGROAS-ProyectoAppWeb/tree/main/entrega-1b)

RESUMEN EJECUTIVO

En la Entrega 1B del proyecto SGROAS se implementó el backend del sistema utilizando Spring Boot 3.x sobre Java 21, con PostgreSQL 18 como motor de base de datos y Flyway para la gestión de migraciones. El desarrollo se enfocó en dos módulos principales: la autenticación de usuarios mediante JSON Web Tokens (JWT) y la gestión de conductores a través de operaciones CRUD completas.

El módulo de autenticación se construyó con Spring Security 6 y la librería jjwt, implementando el registro e inicio de sesión de usuarios, con contraseñas almacenadas mediante hash bcrypt y tokens firmados con el algoritmo HS256, válidos por una hora. Se verificó que las rutas protegidas del sistema rechazan correctamente las solicitudes sin token (403 Forbidden) y aceptan aquellas con un token válido (200 OK).

El módulo de conductores permitió crear, consultar, actualizar y eliminar registros, con persistencia validada directamente en PostgreSQL mediante Spring Data JPA. Todas las operaciones fueron verificadas con Postman, confirmando los códigos de respuesta HTTP esperados (201, 200, 204) y la correcta validación de los datos de entrada, incluyendo el rechazo de solicitudes con información incompleta o mal formada (400 Bad Request).

Los resultados obtenidos confirman que el sistema cuenta con una base de autenticación segura y un módulo funcional de gestión de conductores, estableciendo los cimientos necesarios para la incorporación de los módulos restantes del sistema en las siguientes entregas.

1. INTRODUCCIÓN

1.1 Contexto y motivación

Las cooperativas de transporte interprovincial en Ecuador constituyen un servicio esencial para la movilidad de millones de personas entre ciudades y comunidades rurales. Sin embargo, según un informe de la Agencia Nacional de Tránsito del Ecuador publicado en 2022, más del 60% de estas cooperativas gestionan sus operaciones mediante registros físicos o herramientas ofimáticas no integradas, lo que genera errores administrativos, pérdida de información y dificultades en la supervisión operativa. Adicionalmente, el personal administrativo dedica en promedio más del 40% de su tiempo a tareas de registro manual que podrían automatizarse (Sociedad et al., 2018).

El sistema propuesto busca resolver esta problemática centralizando en una plataforma digital la gestión de vehículos, conductores, rutas, asignaciones, incidentes operativos y reportes de una cooperativa de transporte nacional ecuatoriana.

Los usuarios del sistema son tres: el **Administrador del Sistema**, responsable de gestionar los accesos, usuarios y el registro de auditoría; el **Coordinador de Ruta**, encargado de registrar unidades, conductores, rutas y gestionar sus asignaciones; y el **Personal de Seguridad**, quien registra incidentes operativos y consulta los reportes del sistema.

La solución se implementa como una **aplicación web** porque permite el acceso desde cualquier dispositivo con navegador sin necesidad de instalación, facilita el trabajo simultáneo de varios usuarios desde distintas ubicaciones y reduce los costos de despliegue y mantenimiento frente a una aplicación de escritorio.

Con el propósito de contribuir a la modernización de estos procesos se plantea el desarrollo del Sistema de Gestión de Recursos Operativos, Administrativos y de Seguridad para Cooperativas de Transporte Nacional (SGROAS). Esta solución busca centralizar toda la información institucional dentro de una plataforma web accesible, segura y escalable.

1.2 Objetivos de la entrega

- Implementar el módulo de autenticación basado en JSON Web Tokens (JWT), permitiendo el registro e inicio de sesión seguro de los usuarios del sistema.
- Desarrollar el módulo CRUD (crear, leer, actualizar y eliminar) para la gestión de conductores, utilizando Spring Data JPA sobre PostgreSQL.
- Migrar la base de datos mediante Flyway, garantizando versionamiento y trazabilidad de los cambios en el esquema.
- Validar el funcionamiento de los endpoints desarrollados mediante pruebas manuales mediante Postman, confirmando respuestas y códigos HTTP correctos.

1.3 Alcance de la entrega

Durante la Entrega 1A, el sistema SGROAS fue diseñado como una aplicación web orientada a centralizar la gestión operativa, administrativa y de seguridad de una cooperativa de transporte nacional. En esa fase se priorizó la planificación funcional, los requisitos del sistema, la arquitectura general y el alcance de los módulos principales.

Para la Entrega 1B, la tecnología de implementación fue actualizada conforme a las directrices establecidas por la asignatura, adoptando Java 21, Spring Boot 3.x, Spring Security 6, Spring Data JPA, PostgreSQL 18, Redis y Docker Compose como stack obligatorio. Este cambio no modifica el propósito del sistema ni sus requisitos principales, sino que ajusta la solución a una arquitectura backend más robusta, segura y alineada con el desarrollo de APIs REST empresariales (Sommerville, 2016).

Por ello, los módulos de autenticación y gestión de conductores se implementan ahora sobre Spring Boot, manteniendo la continuidad del diseño documentado previamente.

1.4 Estructura del documento

El presente documento se organiza de la siguiente manera: la sección 4 presenta el marco teórico que sustenta las decisiones técnicas adoptadas; la sección 5 describe el diseño del sistema, incluyendo diagramas y la base de datos; la sección 6 detalla la implementación realizada; la sección 7 expone las pruebas efectuadas; y las secciones finales abordan la seguridad implementada, el historial de control de versiones y las conclusiones del trabajo desarrollado.

2. MARCO TEÓRICO

2.1 Arquitectura de Aplicaciones Web

Las aplicaciones web modernas se desarrollan generalmente mediante arquitecturas multicapa que permiten separar responsabilidades entre los diferentes componentes del sistema. Este enfoque facilita el mantenimiento, la reutilización de código y la escalabilidad de las aplicaciones (Fowler, 2018).

La arquitectura multicapa suele dividirse en capa de presentación, capa de negocio y capa de persistencia. Cada una cumple funciones específicas dentro del flujo de procesamiento de información y contribuye a mantener una estructura organizada del software.

2.2 Spring Boot

Spring Boot es un framework desarrollado sobre el ecosistema Spring que simplifica la construcción de aplicaciones empresariales. Su principal ventaja consiste en reducir la complejidad de configuración y permitir el desarrollo rápido de aplicaciones robustas mediante mecanismos de autoconfiguración.

Actualmente es uno de los frameworks más utilizados para la construcción de APIs REST debido a su flexibilidad, integración con múltiples tecnologías y amplio soporte de la comunidad (Spring Team, 2025).

2.3 Spring Security

Spring Security es un módulo especializado en la protección de aplicaciones desarrolladas con Spring. Proporciona funcionalidades para autenticación, autorización, control de acceso y protección contra amenazas comunes en aplicaciones web.

La integración de Spring Security permite implementar mecanismos de acceso basados en roles y autenticación mediante tokens JWT (Spring Security Team, 2025).

2.4 PostgreSQL

PostgreSQL es un sistema gestor de bases de datos relacional de código abierto ampliamente utilizado en entornos empresariales. Destaca por su estabilidad, seguridad, cumplimiento de estándares SQL y capacidad para gestionar grandes volúmenes de información.

Su robustez lo convierte en una alternativa adecuada para aplicaciones críticas que requieren integridad y disponibilidad de datos (PostgreSQL Global Development Group, 2025).

2.5 Flyway

Flyway es una herramienta utilizada para administrar cambios en bases de datos mediante migraciones versionadas. Cada modificación del esquema se registra mediante scripts SQL organizados secuencialmente.

Su utilización facilita la sincronización de estructuras de base de datos entre distintos integrantes del equipo de desarrollo (Redgate Software, 2025).

2.6 JWT

JSON Web Token (JWT) es un estándar abierto utilizado para transmitir información de autenticación entre cliente y servidor. Los tokens permiten validar la identidad de los usuarios sin necesidad de mantener sesiones tradicionales en el servidor.

Esta tecnología es ampliamente utilizada en arquitecturas REST modernas debido a su eficiencia y facilidad de integración (Jones et al., 2015).

2.7 ORM con Hibernate/JPA

La Persistencia de Java (JPA, *Java Persistence API*) es una especificación que define cómo mapear objetos Java a tablas de una base de datos relacional, permitiendo manipular los datos como entidades en lugar de escribir consultas SQL manuales para cada operación. Hibernate es la implementación más utilizada de esta especificación y constituye el motor de persistencia por defecto en Spring Boot a través del módulo Spring Data JPA.

Mediante anotaciones como `@Entity`, `@Id` y `@Column`, las clases Java se asocian directamente con las tablas de la base de datos, lo que reduce la cantidad de código repetitivo y facilita el mantenimiento del modelo de datos (Spring Data Team, 2025).

Spring Data JPA, además, proporciona repositorios genéricos que automatizan operaciones CRUD comunes sin necesidad de implementarlas manualmente.

2.8 Seguridad JWT según OWASP

El uso de JSON Web Tokens conlleva riesgos de seguridad si no se implementan siguiendo buenas prácticas. OWASP (Open Web Application Security Project) recomienda un conjunto de medidas para mitigar vulnerabilidades comunes asociadas a JWT, entre ellas: validar siempre la firma del token en el servidor antes de confiar en su contenido, establecer tiempos de expiración cortos para reducir la ventana de uso indebido en caso de robo del token, evitar almacenar información sensible dentro del payload (ya que este no está cifrado, solo firmado), y utilizar algoritmos de firma seguros, evitando configuraciones que permitan el algoritmo "none" (OWASP, 2024).

Estas recomendaciones son aplicables directamente al diseño de SGROAS, donde el token emitido contiene únicamente identificadores y rol del usuario, con un tiempo de expiración definido, y donde la validación de la firma se realiza en cada solicitud protegida mediante el filtro de autenticación correspondiente.

3. DISEÑO DEL SISTEMA

El diseño del sistema SGROAS se basa en una arquitectura orientada a servicios implementada mediante Spring Boot. La solución fue concebida para permitir la

administración centralizada de recursos operativos, administrativos y de seguridad dentro de una cooperativa de transporte nacional (Martin, 2018).

La arquitectura implementada sigue el patrón Controller-Service-Repository, ampliamente utilizado en aplicaciones empresariales desarrolladas con Spring. Esta organización permite separar adecuadamente las responsabilidades del sistema y facilita su mantenimiento.

La estructura del backend se encuentra organizada mediante los paquetes config, controller, dto, entity, repository, security y service. Cada paquete cumple una función específica dentro del proyecto y contribuye a mantener una arquitectura modular y escalable; (ver **Figura 4**).

El modelo de datos contempla inicialmente las entidades Usuario y Conductor, las cuales representan la base para la implementación de los módulos de autenticación y gestión operativa. Posteriormente se incorporarán las entidades correspondientes a rutas, unidades, incidentes, reportes y auditoría (Bass et al., 2021).

3.1 Actualización del modelo C4

3.2 Diagrama de clases UML refactorizado

3.3 Tabla de correspondencia Java ↔ PostgreSQL

En la **Tabla 1** se presenta la correspondencia entre los atributos de la entidad Usuario y las columnas correspondientes en PostgreSQL.

Tabla 1. Correspondencia de atributos Java y columnas PostgreSQL – Entidad Usuario

Atributo Java	Tipo Java	Columna PostgreSQL	Tipo PostgreSQL
id	Long	id	BIGSERIAL (PK)
nombre	String	nombre	VARCHAR(100)
email	String	email	VARCHAR(255)
passwordHash	String	password_hash	VARCHAR(255)
rol	String	rol	VARCHAR(30)
activo	Boolean	activo	BOOLEAN
creadoEn	OffsetDateTime	creado_en	TIMESTAMPTZ
actualizadoEn	OffsetDateTime	actualizado_en	TIMESTAMPTZ

La **Tabla 2** muestra la correspondencia equivalente para la entidad Conductor.

Tabla 2. Correspondencia de atributos Java y columnas PostgreSQL – Entidad Conductor

Atributo Java	Tipo Java	Columna PostgreSQL	Tipo PostgreSQL
id	Long	id	BIGSERIAL (PK)
nombres	String	nombres	VARCHAR(100)
apellidos	String	apellidos	VARCHAR(100)
cedula	String	cedula	VARCHAR(10)
numeroLicencia	String	numero_licencia	VARCHAR(30)

tipoLicencia	String	tipo_licencia	VARCHAR(10)
fechaVencimientoLicencia	LocalDate	fecha_vencimiento_licencia	DATE
telefono	String	telefono	VARCHAR(20)
email	String	email	VARCHAR(255)
estado	String	estado	VARCHAR(20)
activo	Boolean	activo	BOOLEAN
creadoEn	OffsetDateTime	creado_en	TIMESTAMPZ
actualizadoEn	OffsetDateTime	actualizado_en	TIMESTAMPZ

3.4 DER y Diccionario de datos

Diagrama Entidad-Relación (DER)

En la **Figura 1** se detalla la estructura lógica de la base de datos, incluyendo las tablas de usuarios y conductores con sus respectivas restricciones de integridad.

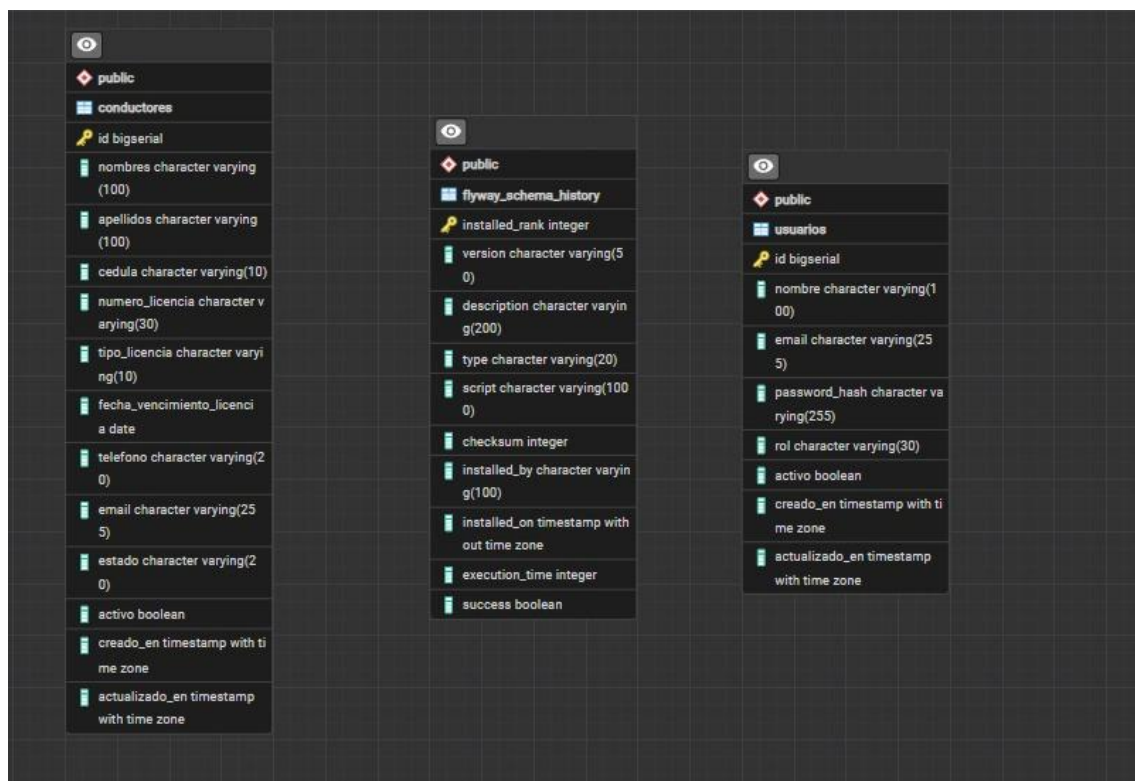


Figura 1. Diagrama Entidad-Relación (DER) del sistema SGROAS

En la **Figura 2** se evidencian los registros almacenados en la base de datos.

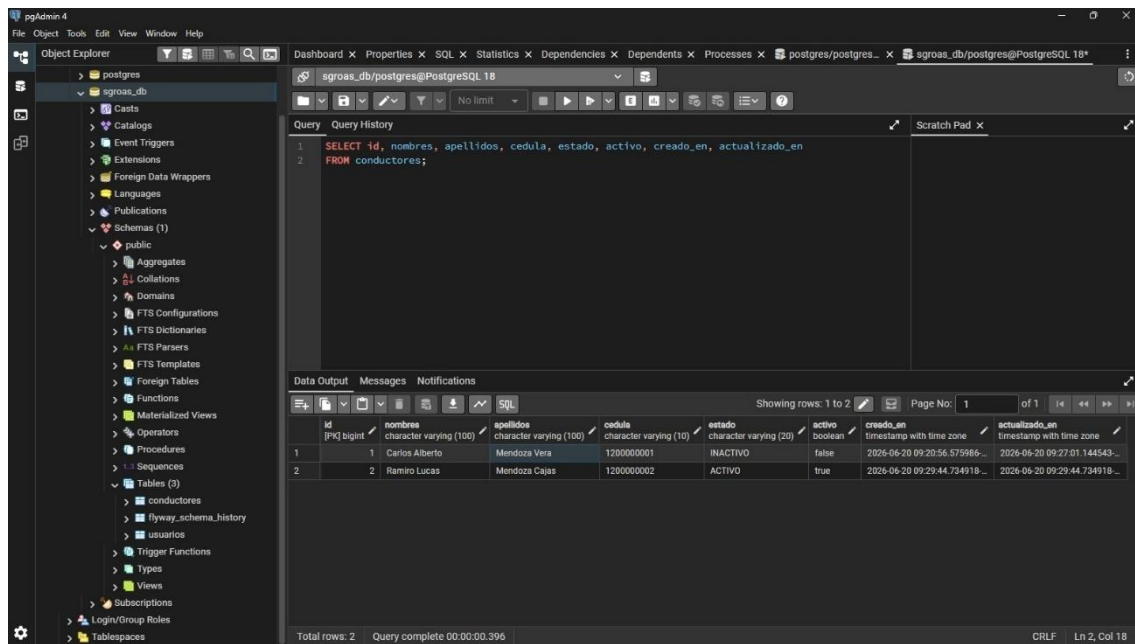


Figura 2. Consulta de los registros almacenados en la tabla conductores (pgAdmin)

Diccionario de datos actualizado

A continuación, la **Tabla 3** detalla las columnas, tipos de datos y restricciones de la tabla usuarios.

Tabla 3. Diccionario de datos – tabla usuarios

Columna	Tipo	Restricciones
id	BIGSERIAL	PK
nombre	VARCHAR(100)	NOT NULL
email	VARCHAR(255)	NOT NULL, único
password_hash	VARCHAR(255)	NOT NULL
rol	VARCHAR(30)	NOT NULL, CHECK (ROLE_ADMIN, ROLE_COORDINADOR, ROLE_SEGURIDAD)
activo	BOOLEAN	NOT NULL, DEFAULT TRUE
creado en	TIMESTAMPTZ	NOT NULL, DEFAULT NOW()
actualizado en	TIMESTAMPTZ	NOT NULL, actualizado por trigger

La **Tabla 4** presenta el diccionario de datos correspondiente a la tabla conductores.

Tabla 4. Diccionario de datos – tabla conductores

Columna	Tipo	Restricciones
id	BIGSERIAL	PK
nombres	VARCHAR(100)	NOT NULL
apellidos	VARCHAR(100)	NOT NULL
cedula	VARCHAR(10)	NOT NULL, único, longitud exacta de 10
numero_licencia	VARCHAR(30)	NOT NULL, único
tipo_licencia	VARCHAR(10)	NOT NULL
fecha_vencimiento_licencia	DATE	NOT NULL, CHECK (>= 2020-01-01)
telefono	VARCHAR(20)	—
email	VARCHAR(255)	—
estado	VARCHAR(20)	NOT NULL, CHECK (ACTIVO, INACTIVO, SUSPENDIDO)
activo	BOOLEAN	NOT NULL, DEFAULT TRUE
creado en	TIMESTAMPTZ	NOT NULL, DEFAULT NOW()

actualizado_en	TIMESTAMPTZ	NOT NULL, actualizado por trigger
----------------	-------------	-----------------------------------

3.5 Diagrama de secuencia: flujo de autenticación JWT

Se muestra a continuación el diagrama de secuencia del flujo de la autenticación de token, como se ve en la **Figura 3**.

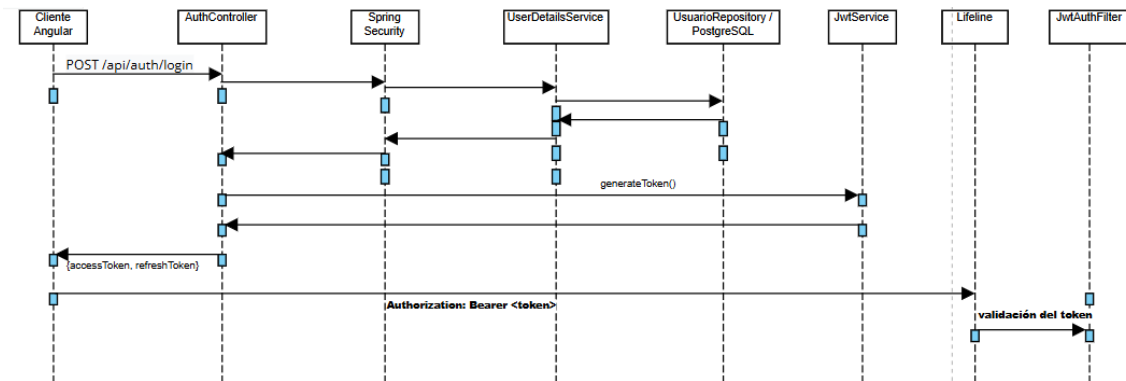


Figura 3. Diagrama de secuencia (autenticación JWT)

3.6 Script Flyway de migración inicial

Se muestra el script desarrollado en el entorno IntelliJ IDEA (véase **Figura 5**).

Código:

```

-- V1__schema_inicial.sql
-- Migracion inicial de SGROAS
-- Entrega 1B: Autenticacion JWT + CRUD de conductores

```

```

CREATE OR REPLACE FUNCTION actualizar_fecha_modificacion()
RETURNS TRIGGER AS $$
BEGIN
    NEW.actualizado_en = NOW();
RETURN NEW;
END;
$$ LANGUAGE plpgsql;

```

```

CREATE TABLE usuarios (
    id BIGSERIAL PRIMARY KEY,
    nombre VARCHAR(100) NOT NULL,
    email VARCHAR(255) NOT NULL,
    password_hash VARCHAR(255) NOT NULL,
    rol VARCHAR(30) NOT NULL DEFAULT 'ROLE_COORDINADOR',
    activo BOOLEAN NOT NULL DEFAULT TRUE,
    creado_en TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    actualizado_en TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

```

```

CREATE UNIQUE INDEX idx_usuarios_email
ON usuarios(email);

```

```

ALTER TABLE usuarios
ADD CONSTRAINT chk_usuarios_rol
CHECK (
    rol IN (
        'ROLE_ADMIN',

```

```
        'ROLE_COORDINADOR',  
        'ROLE_SEGURIDAD'  
    )  
);
```

```
CREATE TRIGGER trg_usuarios_actualizado_en  
    BEFORE UPDATE ON usuarios  
    FOR EACH ROW  
    EXECUTE FUNCTION actualizar_fecha_modificacion();
```

```
CREATE TABLE conductores (  
    id BIGSERIAL PRIMARY KEY,  
    nombres VARCHAR(100) NOT NULL,  
    apellidos VARCHAR(100) NOT NULL,  
    cedula VARCHAR(10) NOT NULL,  
    numero_licencia VARCHAR(30) NOT NULL,  
    tipo_licencia VARCHAR(10) NOT NULL,  
    fecha_vencimiento_licencia DATE NOT NULL,  
    telefono VARCHAR(20),  
    email VARCHAR(255),  
    estado VARCHAR(20) NOT NULL DEFAULT 'ACTIVO',  
    activo BOOLEAN NOT NULL DEFAULT TRUE,  
    creado_en TIMESTAMPTZ NOT NULL DEFAULT NOW(),  
    actualizado_en TIMESTAMPTZ NOT NULL DEFAULT NOW()  
);
```

```
CREATE UNIQUE INDEX idx_conductores_cedula  
    ON conductores(cedula);
```

```
CREATE UNIQUE INDEX idx_conductores_numero_licencia  
    ON conductores(numero_licencia);
```

```
ALTER TABLE conductores  
    ADD CONSTRAINT chk_conductores_estado  
    CHECK (  
        estado IN (  
            'ACTIVO',  
            'INACTIVO',  
            'SUSPENDIDO'  
        )  
    );
```

```
ALTER TABLE conductores  
    ADD CONSTRAINT chk_conductores_cedula_longitud  
    CHECK (char_length(cedula) = 10);
```

```
ALTER TABLE conductores  
    ADD CONSTRAINT chk_conductores_fecha_licencia  
    CHECK (fecha_vencimiento_licencia >= DATE '2020-01-01');
```

```
CREATE TRIGGER trg_conductores_actualizado_en  
    BEFORE UPDATE ON conductores  
    FOR EACH ROW  
    EXECUTE FUNCTION actualizar_fecha_modificacion();
```

4. IMPLEMENTACIÓN

La implementación comenzó con la configuración del entorno de desarrollo utilizando Java 21, Maven y Spring Boot 3.x. Posteriormente se creó la estructura inicial del proyecto siguiendo las buenas prácticas recomendadas para aplicaciones empresariales.

Como parte de esta fase se implementó la organización por paquetes que incluye las capas de configuración, controladores, servicios, repositorios, entidades, seguridad y objetos de transferencia de datos, (véase **Figura 4**).

La primera migración desarrollada permitió la creación de las tablas usuarios y conductores. Estas estructuras incluyen restricciones de integridad, índices únicos y mecanismos automáticos de actualización mediante triggers, (véase **Figura 5**).

Asimismo, se configuró PostgreSQL como sistema gestor de bases de datos y Flyway como herramienta para el control de versiones del esquema de persistencia.

Adicionalmente, se preparó la infraestructura necesaria para implementar autenticación basada en JWT y autorización mediante roles utilizando Spring Security (PostgreSQL Global Development Group, 2025). Véase en la **Figura 6**, las tablas fueron creadas correctamente en el esquema de la base de datos.

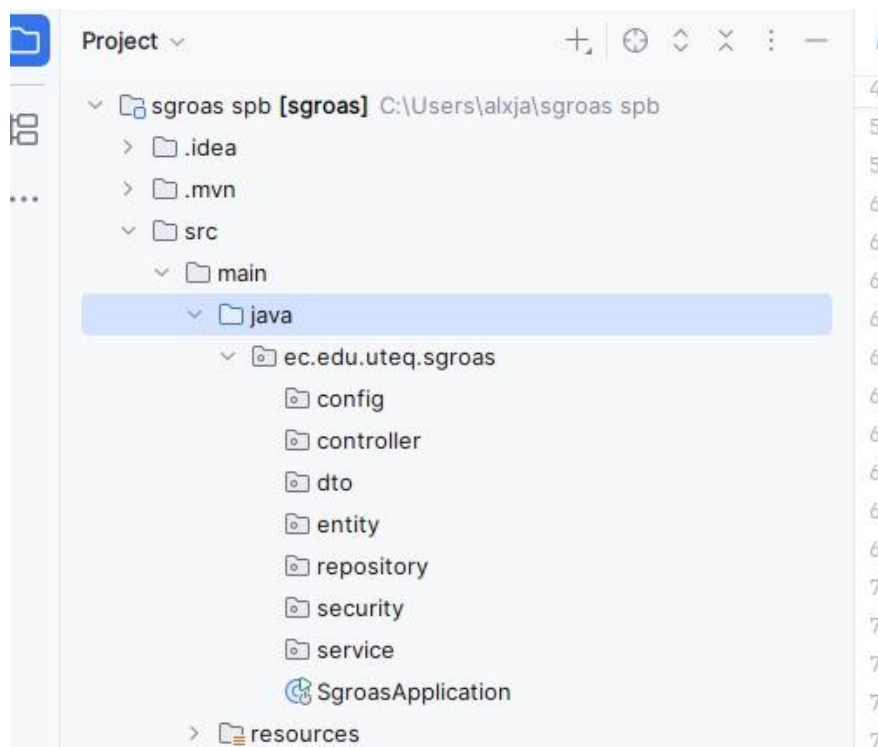


Figura 4. Estructura de paquetes del backend Spring Boot

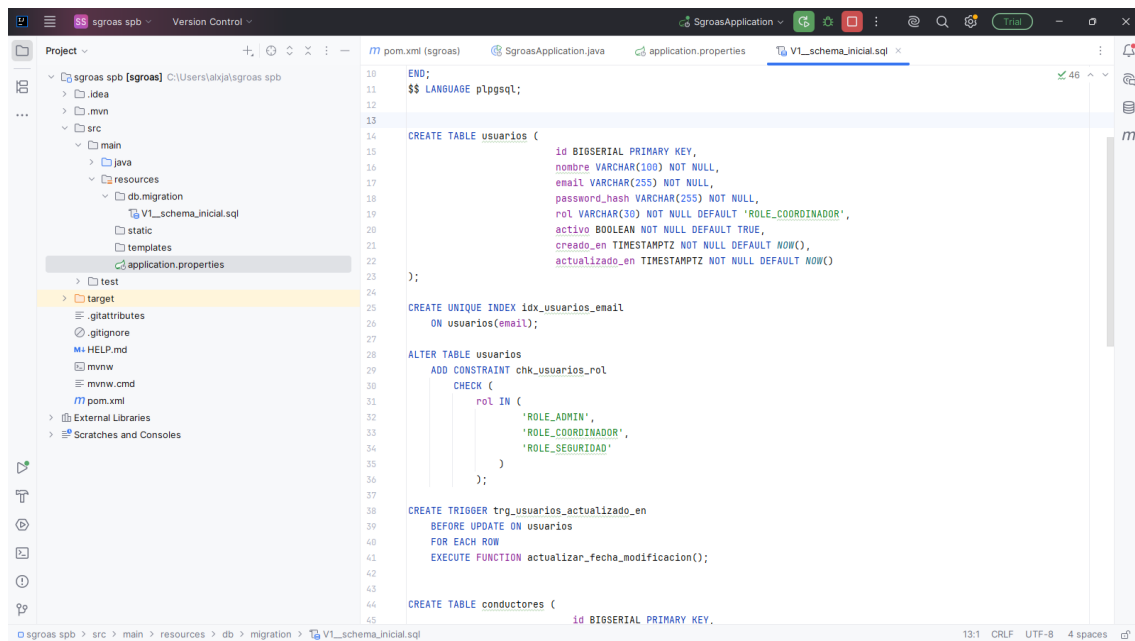


Figura 5. Script de migración inicial (V1__schema_inicial.sql) en el entorno de desarrollo

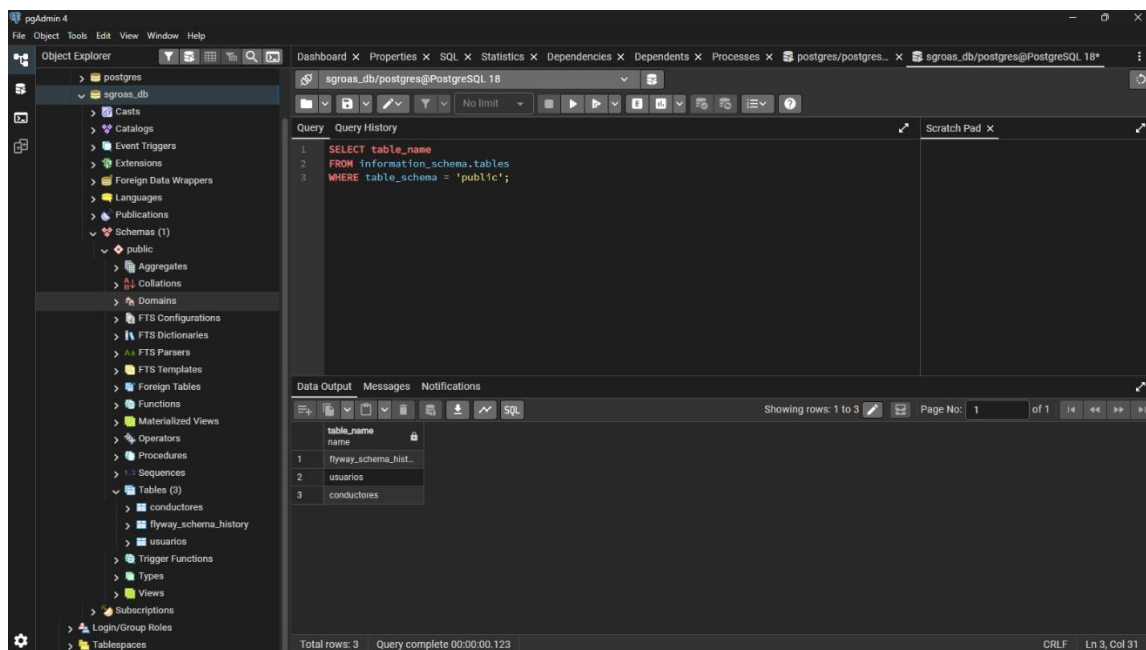


Figura 6. Verificación de las tablas creadas en PostgreSQL mediante pgAdmin

4.1 Estructura del repositorio monorepo

4.2 Configuración de Spring Boot

Se muestra a continuación como se tiene organizada la configuración de spring boot, como se ve en la **Figura 7**.

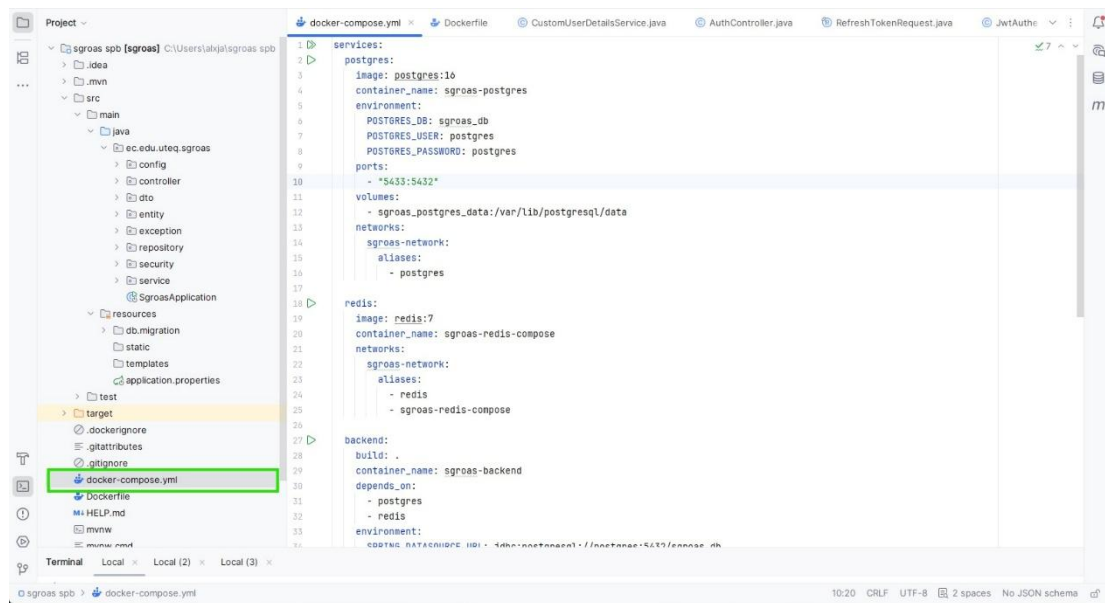


Figura 7. Configuración de Spring Boot

4.3 Implementación del filtro JWT

La validación de los tokens se implementó mediante Spring Security 6 en conjunto con la librería **jjwt**, encargada de firmar y verificar los tokens emitidos. El filtro de autenticación intercepta cada solicitud entrante y distingue entre rutas públicas y rutas protegidas: los endpoints bajo `/api/auth/**` (registro e inicio de sesión) son accesibles sin token, mientras que el resto de endpoints, como `/api/conductores`, requieren la presencia de un token válido en el encabezado Authorization con el formato Bearer <token>.

Cuando una solicitud llega sin token o con un token inválido a una ruta protegida, el sistema responde con código **403 Forbidden**, impidiendo el acceso al recurso. Por el contrario, cuando el token es válido y se envía correctamente en el encabezado, la solicitud es procesada con normalidad, devolviendo el recurso solicitado con código **200 OK**.

4.4 Endpoints implementados

En la **Tabla 5** se listan los endpoints implementados junto con su método HTTP, descripción funcional y código de respuesta verificado.

Tabla 5. Endpoints implementados en los módulos de autenticación y gestión de conductores

Método	URL	Descripción	Código HTTP verificado
POST	/api/auth/register	Registra un nuevo usuario en el sistema	201 Created
POST	/api/auth/login	Autentica al usuario y devuelve un token JWT	200 OK
POST	/api/conductores	Crea un nuevo conductor	201 Created
POST	/api/auth/refresh	Genera un nuevo par de tokens (access y refresh) a partir de un refresh token válido	200 OK
POST	/api/auth/logout	Cierra la sesión del usuario invalidando su token actual	204 No Content
GET	/api/conductores	Lista los conductores registrados (paginado)	200 OK
PUT	/api/conductores/{id}	Actualiza los datos de un conductor existente	200 OK
DELETE	/api/conductores/{id}	Elimina un conductor por su identificador	204 No Content

Todos los endpoints fueron verificados mediante Postman, confirmando tanto el código de respuesta HTTP como la estructura del cuerpo (JSON) devuelto en cada caso. Adicionalmente, se comprobó que el acceso al endpoint /api/conductores sin un token válido es rechazado por el sistema con código **403 Forbidden**, confirmando que la ruta está protegida correctamente.

4.5 Estructura del JWT emitido

El token generado por el sistema utiliza el algoritmo de firma **HS256** (HMAC con SHA-256), proporcionado por la librería jjwt. Su tiempo de vida es de **una hora** (3,600,000 milisegundos), tal como se evidencia en el campo expiresIn devuelto en las respuestas de registro y login.

Además del propio token (accessToken), la respuesta del servidor incluye información complementaria útil para el cliente: el tipo de token (tokenType: "Bearer"), el nombre y correo del usuario autenticado, y su rol asignado (por ejemplo, ROLE_ADMIN). Esto permite que el frontend identifique al usuario sin necesidad de decodificar el token manualmente en esta etapa del desarrollo.

4.6 Implementación Angular: interceptor JWT

5. PRUEBAS AUTOMATIZADAS

Durante las siguientes iteraciones del proyecto se desarrollarán pruebas automatizadas utilizando JUnit 5 y Spring Boot Test (Pressman & Maxim, 2020).

Las pruebas estarán orientadas a validar el comportamiento de controladores, servicios, repositorios y mecanismos de autenticación implementados dentro del sistema (Sommerville, 2016).

5.1 Pruebas unitarias del servicio de autenticación

Se implementaron pruebas unitarias para validar el comportamiento de los servicios de autenticación y de gestión de conductores, utilizando JUnit 5 y Mockito.

Para el servicio de autenticación (AuthServiceTest), se verificó que el inicio de sesión con credenciales correctas retorne exitosamente un access token y un refresh token, y que el inicio de sesión con contraseña incorrecta lance una excepción de credenciales inválidas.

Para el servicio de conductores (ConductorServiceTest), se verificó la creación correcta de un conductor, el rechazo de una cédula duplicada mediante una excepción controlada, y la búsqueda de un conductor por su identificador. La **Tabla 6** resume los casos de prueba ejecutados y su resultado.

Tabla 6. Resultados de las pruebas unitarias ejecutadas con JUnit 5 y Mockito

Clase de prueba	Caso validado	Resultado
AuthServiceTest	Login correcto con generación de tokens	Aprobado

AuthServiceTest	Login con contraseña incorrecta	Aprobado
ConductorServiceTest	Crear conductor correctamente	Aprobado
ConductorServiceTest	Validar cédula duplicada	Aprobado
ConductorServiceTest	Buscar conductor por ID	Aprobado

La ejecución conjunta de ambas clases de prueba resultó en 5 pruebas aprobadas de 5 ejecutadas, cumpliendo con el mínimo de pruebas unitarias requerido para esta entrega.

La correcta ejecución y aprobación de la lógica de negocio para los servicios de autenticación y conductores se documenta en las **Figura 8** y **Figura 9**.

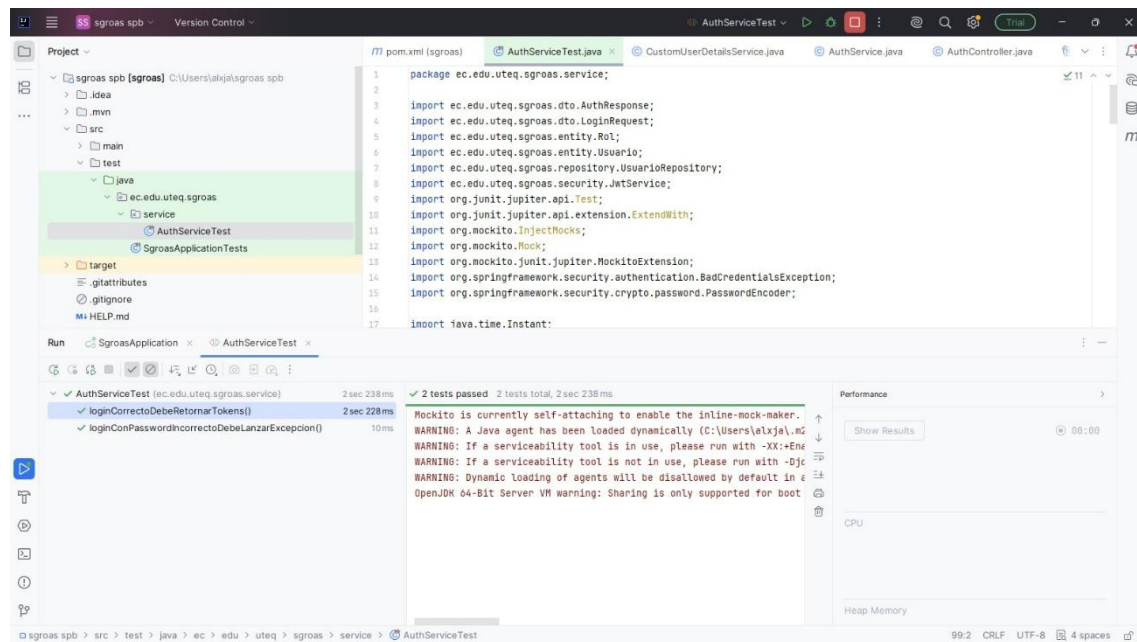


Figura 8. Resultado de ejecución de pruebas unitarias para AuthServiceTest mediante JUnit 5

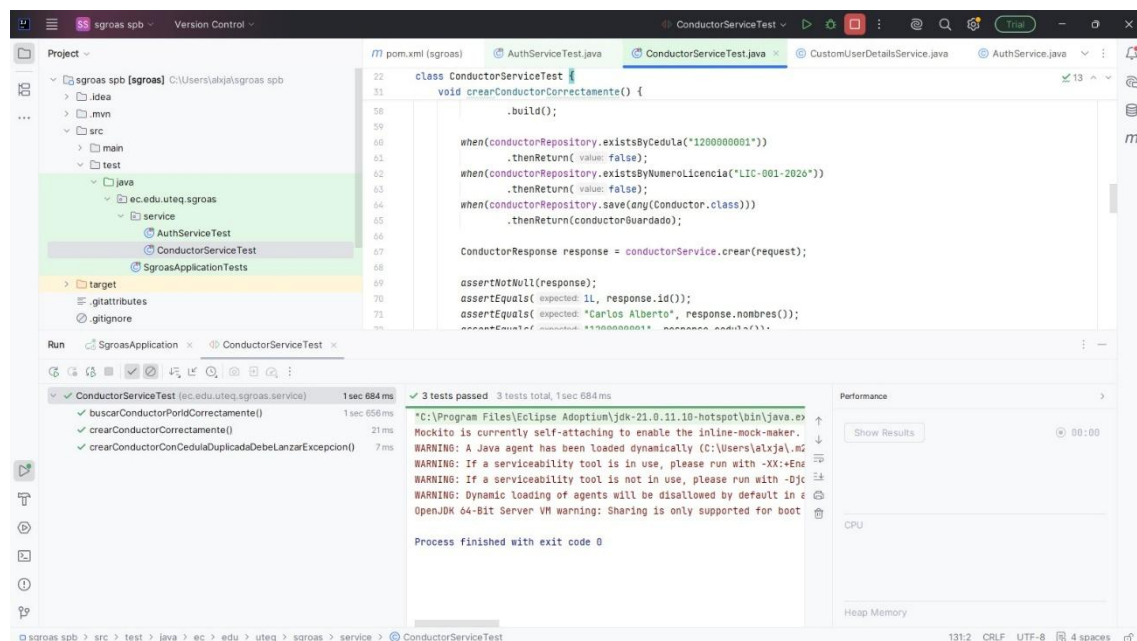


Figura 9. Resultado de ejecución de pruebas unitarias para ConductorServiceTest mediante JUnit 5

5.2 Pruebas de integración con Postman

Se realizaron pruebas funcionales sobre los endpoints de autenticación y de gestión de conductores, utilizando Postman como herramienta de verificación. Los casos de prueba ejecutados fueron los siguientes (ver **Tabla 7**):

***Tabla 7.** Resultados de las pruebas de integración ejecutadas con Postman*

Caso de prueba	Endpoint	Resultado esperado	Resultado obtenido
Registro de usuario con datos válidos	POST /api/auth/register	201 Created	201 Created
Registro de usuario con datos inválidos (nombre vacío, email mal formado, contraseña corta, rol vacío)	POST /api/auth/register	400 Bad Request con detalle de errores	400 Bad Request
Inicio de sesión con credenciales correctas	POST /api/auth/login	200 OK con token JWT	200 OK
Inicio de sesión con contraseña incorrecta	POST /api/auth/login	401 Unauthorized	401 Unauthorized
Renovación de tokens con refresh token válido	POST /api/auth/refresh	200 OK con nuevo par de tokens	200 OK
Cierre de sesión con token válido	POST /api/auth/logout	204 No Content	204 No Content
Acceso a recurso protegido sin token	GET /api/conductores	403 Forbidden	403 Forbidden
Acceso a recurso protegido con token válido	GET /api/conductores	200 OK	200 OK
Creación de conductor	POST /api/conductores	201 Created	201 Created
Actualización de conductor	PUT /api/conductores/{id}	200 OK	200 OK
Eliminación de conductor	DELETE /api/conductores/{id}	204 No Content	204 No Content

En la **Figura 10** se evidencia el registro exitoso de un usuario, en la **Figura 11** se muestra las pruebas de POST /api/auth/refresh, en la **Figura 12** se muestra el cierre de sesión mediante POST /api/auth/logout. También en la **Figura 13**, se confirma que la contraseña fue almacenada de forma segura mediante hash. La **Figura 14** muestra la respuesta del sistema ante un inicio de sesión válido y como se evidencia en la **Figura 15**, el sistema rechaza el acceso cuando no se proporciona un token válido.

El acceso exitoso a recursos protegidos mediante el uso de un token Bearer válido se confirma en la **Figura 16**. El sistema garantiza la integridad de los datos rechazando registros incompletos o mal formados, como se ilustra en la **Figura 17**. El control de acceso deniega el ingreso ante credenciales de autenticación erróneas, lo cual se evidencia en la **Figura 18**.

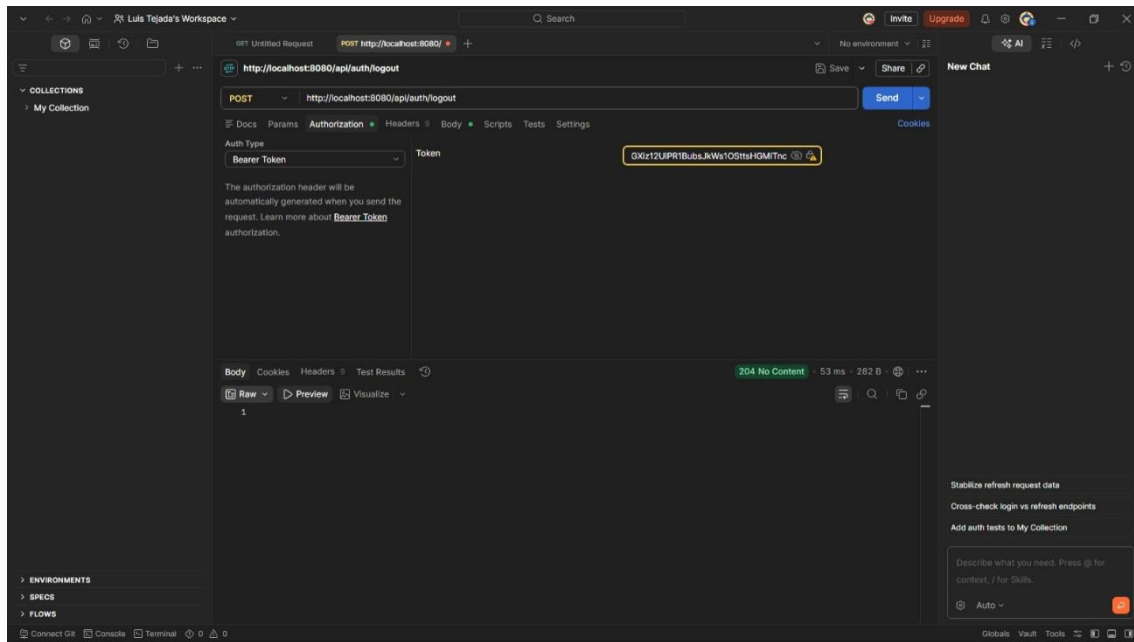


Figura 12. Cierre de sesión exitoso mediante el endpoint POST /api/auth/logout

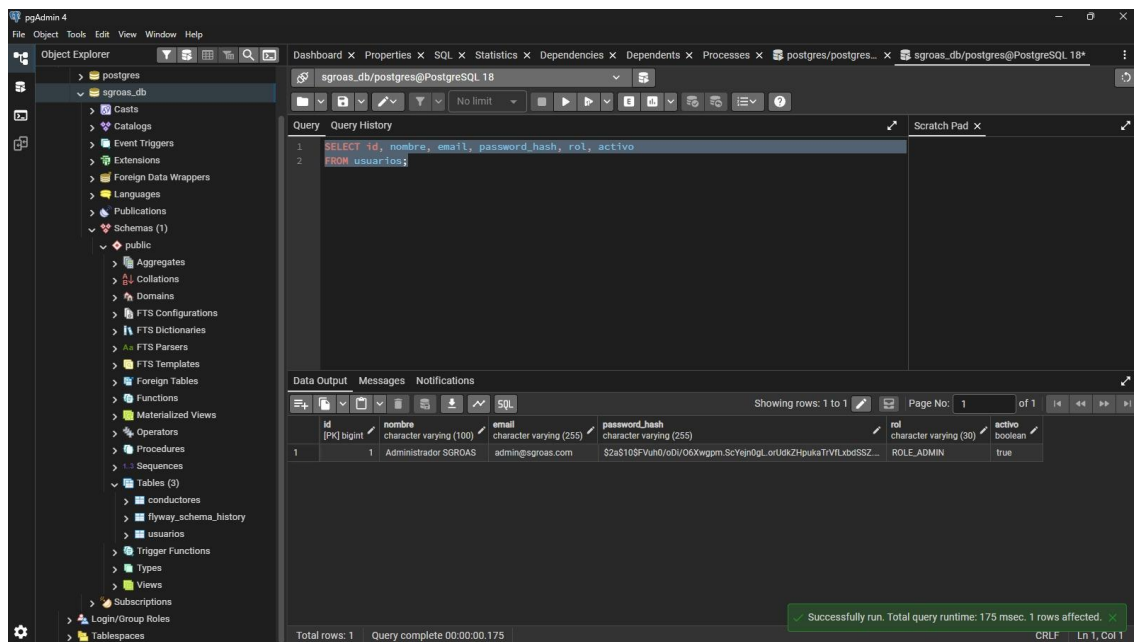
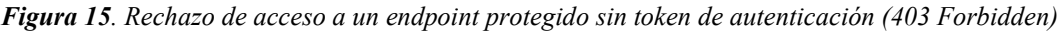
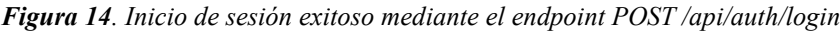


Figura 13. Verificación del registro del usuario en PostgreSQL (contraseña almacenada mediante hash bcrypt)



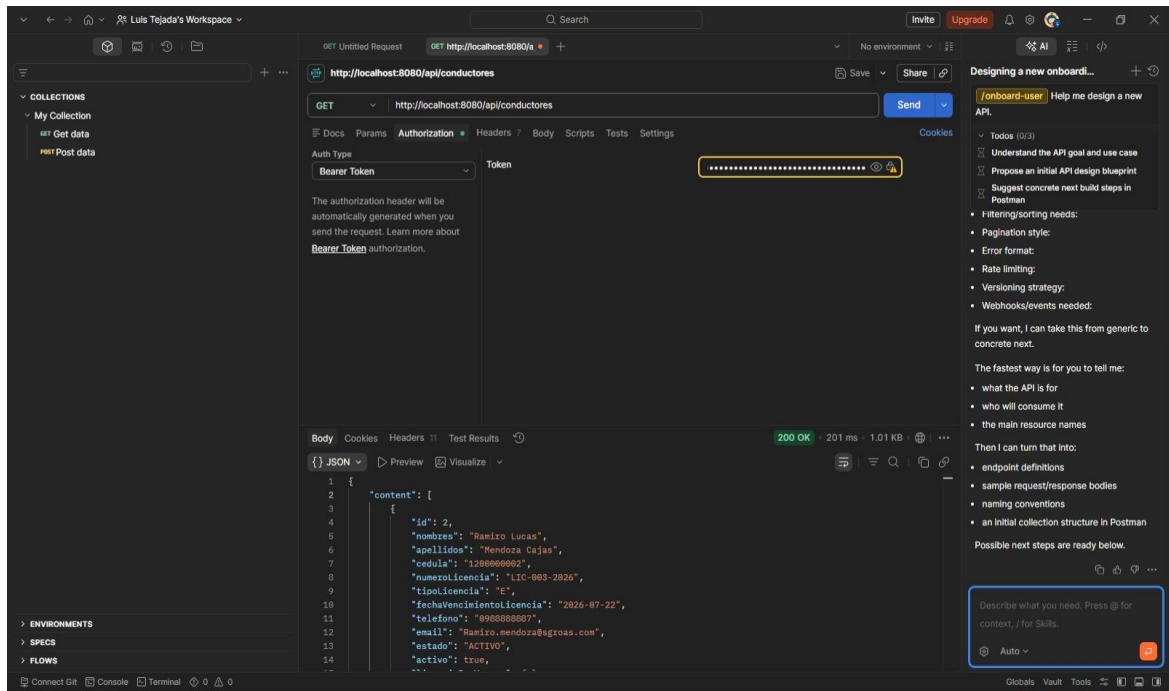


Figura 16. Acceso autorizado a un endpoint protegido mediante token Bearer válido (200 OK)

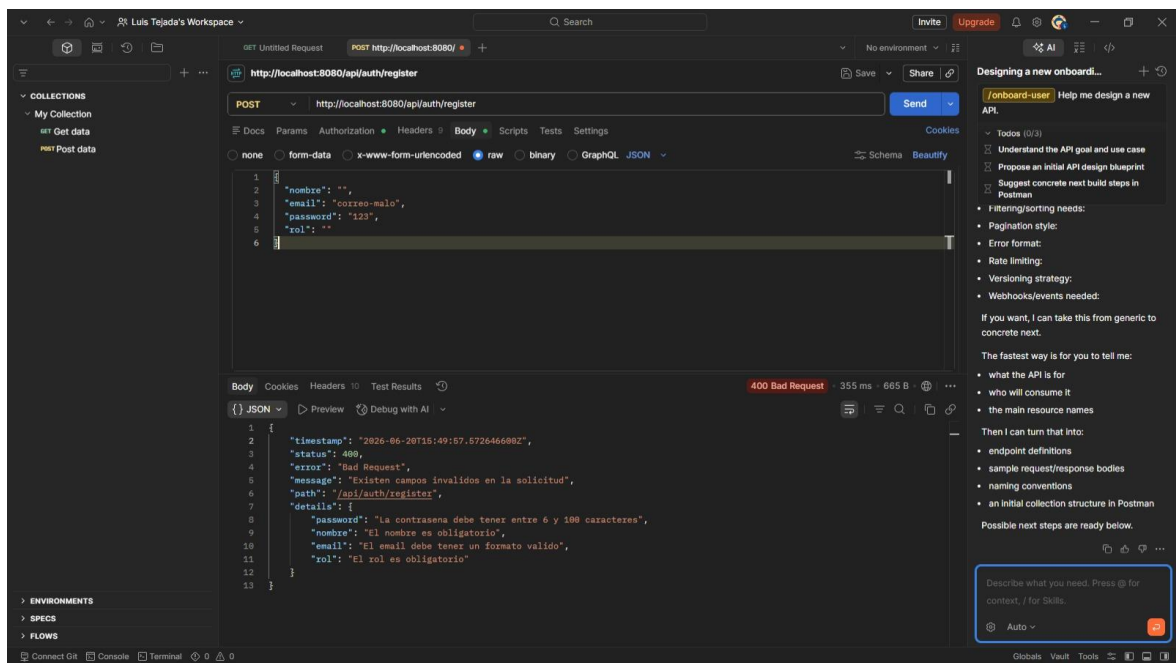


Figura 17. Validación de datos inválidos en el registro de usuario (400 Bad Request)

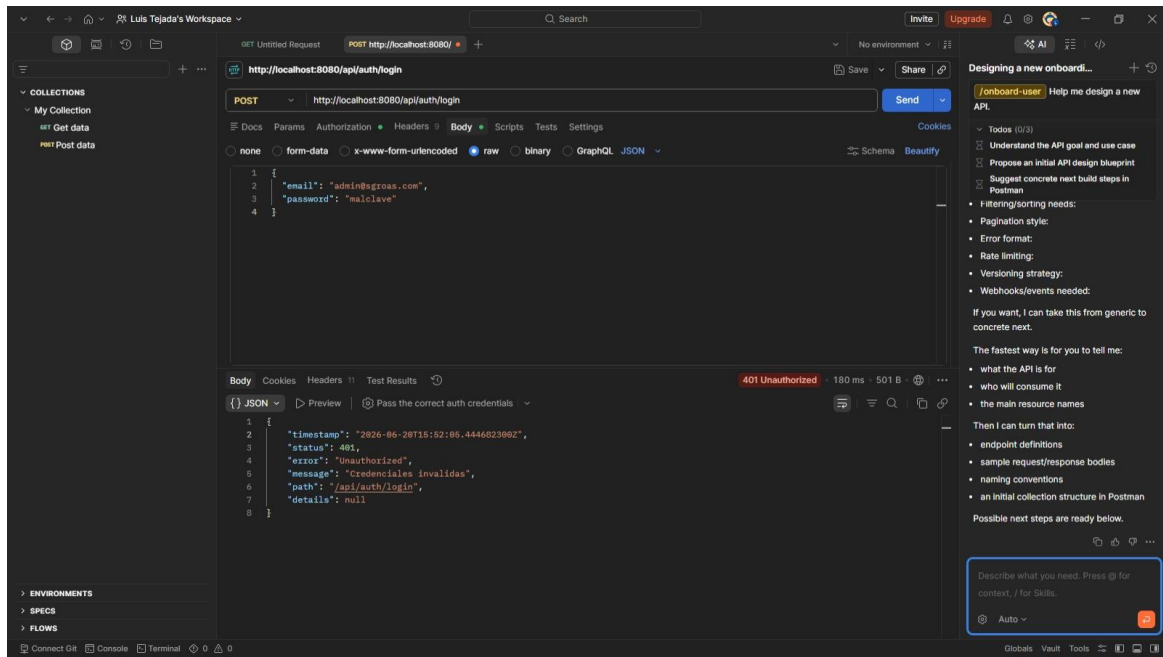


Figura 18. Rechazo de inicio de sesión con credenciales inválidas (401 Unauthorized)

Las pruebas confirmaron que el sistema valida correctamente los datos de entrada, protege las rutas sensibles mediante autenticación JWT, y ejecuta las operaciones CRUD de conductores de forma consistente con los códigos de respuesta HTTP esperados.

5.3 Métricas de rendimiento

Las métricas de rendimiento permitirán evaluar el comportamiento del sistema bajo diferentes condiciones de carga y uso (Kleppmann, 2017).

Entre los indicadores considerados se encuentran tiempo de respuesta de la API, rendimiento de consultas a PostgreSQL, consumo de memoria y utilización de recursos del servidor (Tanenbaum & Steen, 2021).

6. SEGURIDAD IMPLEMENTADA

La seguridad constituye uno de los pilares fundamentales del sistema SGROAS debido a la naturaleza sensible de la información gestionada por la plataforma.

Para minimizar riesgos se consideran las recomendaciones establecidas por OWASP (Open Worldwide Application Security Project), organización internacional especializada en seguridad de aplicaciones web.

Entre las principales vulnerabilidades identificadas por OWASP se encuentran la pérdida de control de acceso, fallas criptográficas, inyección de código, configuraciones inseguras y errores de autenticación.

Para mitigar estos riesgos se implementan mecanismos de autenticación mediante JWT, control de acceso basado en roles, validación de datos de entrada y almacenamiento seguro de contraseñas utilizando algoritmos hash.

Asimismo, Spring Security proporciona herramientas para reforzar la protección de los recursos expuestos por la API y garantizar que únicamente usuarios autorizados puedan acceder a las funcionalidades correspondientes (OWASP Foundation, 2025).

En esta entrega se incorporaron las siguientes medidas de seguridad:

- **Hash de contraseñas:** las contraseñas de los usuarios no se almacenan en texto plano; se guardan mediante un hash (bcrypt), tal como se evidencia en la columna `password_hash` de la tabla usuarios.
- **Autenticación basada en tokens:** el acceso a los recursos protegidos requiere un token JWT válido, firmado con el algoritmo HS256, con una expiración de una hora.
- **Protección de rutas:** los endpoints de gestión de conductores están restringidos mediante Spring Security, devolviendo 403 Forbidden ante solicitudes sin autenticación.
- **Validación de entradas:** los datos recibidos en el registro de usuarios son validados antes de procesarse (campos obligatorios, formato de email, longitud mínima de contraseña), devolviendo 400 Bad Request con el detalle del error cuando no se cumplen las reglas.
- **Manejo controlado de errores de autenticación:** ante credenciales incorrectas, el sistema responde con 401 Unauthorized y un mensaje genérico ("Credenciales inválidas"), sin revelar si el error fue en el email o en la contraseña, evitando dar pistas a un posible atacante.
- **Rotación de tokens y cierre de sesión controlado:** además del token de acceso (`accessToken`), el sistema emite un `refreshToken` que permite renovar la sesión sin requerir un nuevo inicio de sesión. Al cerrar sesión (`/api/auth/logout`), el token se invalida mediante Redis, impidiendo su reutilización posterior.

Como resumen, en la implementación de seguridad se configuró un mecanismo de autenticación basado en JWT, utilizando Spring Security 6 y la librería `jjwt`.

El sistema permite registrar usuarios, iniciar sesión y generar un token de acceso firmado con el algoritmo HS256. Este token tiene una duración limitada de una hora y debe enviarse en las solicitudes protegidas mediante el encabezado `Authorization` con el formato `Bearer Token`.

Para validar su funcionamiento, se realizaron pruebas en Postman, donde se comprobó que los endpoints públicos de autenticación permiten el registro e inicio de sesión, mientras que los endpoints protegidos, como la gestión de conductores, requieren un token válido para ser utilizados.

7. DOCKER COMPOSE Y DESPLIEGUE

Docker Compose será utilizado para facilitar la ejecución coordinada de los servicios que conforman el sistema.

La arquitectura contempla contenedores independientes para la aplicación Spring Boot, PostgreSQL y Redis. Este enfoque permite simplificar la configuración de entornos de desarrollo y despliegue.

La configuración definitiva será incorporada una vez concluida la integración completa de los servicios (Docker Inc., 2025).

Se incorporó Redis como contenedor mediante Docker, utilizado para el manejo de tokens (refresh tokens y posible invalidación en logout). Por el momento la ejecución se realizó de forma manual mediante `docker run --name sgroas-redis -p 6379:6379 -d redis:7`.

En la **Figura 19** se muestra la ejecución del comando `docker run` para desplegar Redis, así como en la **Figura 20** se visualiza que el contenedor `sgroas-redis` se encuentra activo, confirmándose así su correcto funcionamiento mediante `docker ps`.



Figura 19. Ejecución del contenedor Redis mediante Docker



Figura 20. Verificación del contenedor Redis en ejecución (docker ps)

8. HISTORIAL DE GIT

El control de versiones del proyecto se realiza mediante Git y GitHub.

Esta herramienta permite registrar cada modificación realizada por los integrantes del equipo, mantener un historial completo de cambios y facilitar el trabajo colaborativo.

El repositorio centralizado garantiza la disponibilidad del código fuente y facilita la integración de nuevas funcionalidades mediante ramas, commits y procesos de sincronización.

La utilización de Git contribuye además a la trazabilidad del desarrollo y a la recuperación de versiones anteriores cuando sea necesario.

9. CONCLUSIONES

Respecto al objetivo de implementar la autenticación basada en JWT, se logró desarrollar un mecanismo funcional de registro e inicio de sesión, en el cual las contraseñas se almacenan de forma segura mediante hash y los tokens emitidos cuentan con firma digital y tiempo de expiración definido. Uno de los principales desafíos durante esta etapa fue garantizar que las rutas protegidas del sistema rechazaran correctamente las solicitudes no autenticadas sin afectar el acceso de los usuarios válidos; esto se resolvió mediante la

configuración adecuada de los filtros de seguridad de Spring Security, diferenciando explícitamente las rutas públicas de las protegidas.

En cuanto al desarrollo del módulo CRUD de conductores, se cumplió satisfactoriamente el objetivo de permitir la creación, consulta, actualización y eliminación de registros, apoyándose en Spring Data JPA para la comunicación con la base de datos. La validación de los datos de entrada representó un punto importante de atención, ya que fue necesario definir reglas claras para campos como la cédula, el número de licencia y la fecha de vencimiento, evitando así el ingreso de información inconsistente.

La migración de la base de datos mediante Flyway permitió mantener un control ordenado y versionado del esquema, facilitando que los cambios estructurales pudieran aplicarse de manera reproducible. Esto resultó especialmente útil para sincronizar el trabajo entre los integrantes del equipo sin generar conflictos en la estructura de las tablas.

Finalmente, la validación de los endpoints mediante Postman permitió confirmar el correcto funcionamiento de las operaciones implementadas, tanto en los casos de éxito como en los escenarios de error controlado, brindando confianza sobre la estabilidad del backend desarrollado hasta este punto del proyecto.

REFERENCIAS:

- Bass, L., Clements, P., & Kazman, R. (2021). *Software Architecture in Practice* (4th ed.). Addison-Wesley.
- Docker Inc. (2025). *Docker Compose Documentation*.
- Fowler, M. (2018). *Patterns of Enterprise Application Architecture*. Addison-Wesley.
- Jones, M., Bradley, J., & Sakimura, N. (2015). JSON Web Token (JWT). *RFC 7519*.
- Kleppmann, M. (2017). *Designing Data-Intensive Applications*. O'Reilly Media.
- Martin, R. C. (2018). *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. Prentice Hall.
- OWASP Foundation. (2025). *OWASP Top 10: Web Application Security Risks*.
- OWASP. (2024). *JSON Web Token (JWT) Cheat Sheet for Java*.
https://cheatsheetseries.owasp.org/cheatsheets/JSON_Web_Token_for_Java_Cheat_Sheet.html
- PostgreSQL Global Development Group. (2025). *PostgreSQL Documentation*.
- Pressman, R. S., & Maxim, B. R. (2020). *Software Engineering: A Practitioner's Approach* (9th ed.). McGraw-Hill Education.
- Redgate Software. (2025). *Flyway Documentation*.

Sociedad, U. Y., Christian, M., Moreno Rodríguez, J., Melvin, M., Franco, L. L., Rodríguez, M., & Franco, L. (2018). *Volumen 10 | Número 5 | Octubre-Diciembre*. <http://rus>.

Sommerville, I. (2016). *Software Engineering* (10th ed.). Pearson.

Spring Data Team. (2025). *Spring Data JPA*. <https://docs.spring.io/spring-data/jpa/reference/>

Spring Security Team. (2025). *Spring Security Reference Documentation*.

Spring Team. (2025). *Spring Data JPA Documentation*.

Tanenbaum, A. S., & Steen, M. Van. (2021). *Distributed Systems* (4th ed.). Pearson.